

FFT Audio Spectrum Analyzer IP – English version

Audio spectrum-analyzer IP built on a **pipelined R2SDF 256-point FFT**, Q1.15, AXI4-Stream, simulation-only. The RTL core is logic-verified against a cycle-accurate Python model (bit-exact vs golden, SQNR ~60 dB, gap-tolerant); it is **not HDL-compiled in the file-creation environment** – run the testbench in Vivado to confirm syntax/timing.

1. Overview

- **Architecture:** R2SDF DIF, N=256, fs=48 kHz, Q1.15 16-bit, 1 sample/cycle throughput, **bit-reversed** output (reordered to natural, single-sided N/2=128 bins by `bin_ram_output`).
- **Chain:** audio (AXI-S) → `input_buffer_fsm` → `window_unit` (Hann) → `fft_r22sdf_top` → `magnitude_unit` (α -max- β -min) → `bin_ram_output` → `peak_detector`.
- **Bin → frequency:** $f = k \cdot fs / N$ (187.5 Hz/bin).

2. Completeness (vs PDF §2 module list)

Group	Modules	Status
FFT core	<code>butterfly_r2_stage</code> , <code>twiddle_rom</code> , <code>complex_multiplier</code> , <code>stage_with_twiddle</code> , <code>fft_r22sdf_top</code>	✓ (sel_o/cnt_o bug fixed; 3-cyc Karatsuba; sync ROM; valid-gating)
Pre/post	<code>input_buffer_fsm</code> , <code>window_unit</code> , <code>window_rom</code> , <code>magnitude_unit</code> , <code>peak_detector</code>	✓ (team)
Reorder	<code>bin_ram_output</code> (magnitude), <code>bit_reverse_reorder</code> (complex, optional)	✓ (new)
IP boundary	<code>axi_stream_slave_if</code> , <code>axi_stream_master_if</code>	✓ (new, skid buffers)
Control	<code>fft_control_fsm</code> (status/enable)	✓ (new)
Integration	<code>project_top</code> , <code>fft_analyzer_ip</code> (top IP, AXI-S both ends)	✓
Python	<code>golden_model_fft</code> , <code>gen_twiddle_rom</code> , <code>gen_window_coeff</code> , <code>gen_vectors</code> , <code>compare_sqnr</code>	✓
Testbench	<code>tb_fft_r22sdf</code> (core), <code>tb_top.sv</code> (system)	✓
Spec	<code>interface_spec.md</code>	✓

`scaling_schedule_ctrl` and `convergent_rounding` are **integrated** (the `SCALE_EN` parameter and per-module rounding), matching the intent stated in PDF §2.

3. Key design points

- **Delay-commutator butterfly** with an internal delay line (`DELAY_LEN=128...1`); it exports **registered** `sel_o` / `cnt_o` aligned to `out_re/out_im` (the W-phase bug fix).
- **4-cycle twiddle branch** (sync ROM 1 + Karatsuba 3); **4-cycle delay-matched sum bypass**; MUX on `sel_o` delayed by 4 → no phase skew.
- **÷2 per stage** (total 1/N) prevents Q1.15 overflow → SQNR ~60 dB.
- **Valid-gating** (en port) tolerates the 1-cycle inter-frame bubble of `input_buffer_fsm`.
- Core latency = **283 valid-samples** (measured; the testbench re-measures it).

4. Build & simulate on Vivado

```
python3 python/gen_twiddle_rom.py --n 256 --out rom_out
python3 python/gen_window_coeff.py
python3 python/gen_vectors.py
# core check
iverilog -g2012 -o sim rtl/tb_fft_r22sdf.v rtl/fft_r22sdf_top.v rtl/stage_with_twiddle.v \
    rtl/butterfly_r2_stage.v rtl/complex_multiplier.v rtl/twiddle_rom.v && vvp sim
# system check (Vivado xsim, SystemVerilog): add all rtl/*.v + tb_top.sv, run tb_top
```

Expected `tb_fft_r22sdf` : high SQNR, peak bin=10, **PASS**, prints measured latency (set `LATENCY / FFT_LATENCY` to it if ≠283). `tb_top` : peak bin=10, **PASS**.

5. Verilog ↔ Python verification map

```
complex_multiplier ↔ cmul_q15; twiddle_rom ↔ gen_twiddle_rom / twiddle_q15; butterfly_r2_stage ↔ _Stage / round_sat_shift;
fft_r22sdf_top ↔ fft_core_frame (SQNR); magnitude_unit ↔ α-max-β-min; bin_ram_output ↔ bitrev; whole chain ↔ compare_sqnr.py.
```

6. Notes

- If the original `complex_multiplier` lacks `en`: (a) add `en`, or (b) guarantee continuous valid into the FFT. If its multiplier pipeline ≠ 3 stages → adjust bypass depth + LATENCY.
- The spectrum output is self-timed; keep `m_axis_tready` high or add a FIFO before `axi_stream_master_if`.

See `interface_spec.md` for full port/format specification.